Ticket #10 – central logs on stor01 · final summary

Project 2 – Acme Analytics Linux platform · Phase 5, Step 5.4 · rsyslog · 11 Nov 2026 · ✔ solved

1 · The map – which file is on which server

app01 · sender
`90-forward.conf`
send every line to stor01, with a queue

stor01 · receiver
`10-receive.conf`
listen on TCP 514, save per server + program

```
/var/log/remote/app01/ticket10.log
/var/log/remote/app01/price-update.log
/var/log/remote/db01/ticket10.log
/var/log/remote/db01/postgres.log
```

db01 · sender
`90-forward.conf` (same as app01)
`postgresql.conf`
DB log also to rsyslog

app01 ──TCP 514──▶ stor01 ◀──TCP 514── db01 · **You write 3 settings files. rsyslog makes all log folders and files by itself** (folder = server name, file = program name or `logger -t` tag).

2 · The 3 settings files

stor01 – receive

stor01 · the file as typed in nano (yellow = the important parts)

```
GNU nano 6.2                                  /etc/rsyslog.d/10-receive.conf
module(load="imtcp")
input(type="imtcp" port="514" ruleset="remote")
template(name="PerHost" type="string"
         string="/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log")
ruleset(name="remote") {
  action(type="omfile" dynaFile="PerHost")
  stop
}
^O Write Out (save)    ^X Exit
```

`imtcp` = **i**nput **m**odule (receive) · `port="514"` = the door · `string="/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"` = where to save · `omfile` = **o**utput **m**odule, write to file · `stop` = nowhere else

app01 and db01 – send

app01 and db01 · the same file typed in nano on both (yellow = the important parts)

```
GNU nano 6.2                                  /etc/rsyslog.d/90-forward.conf
*.* action(type="omfwd" target="192.168.60.20" port="514" protocol="tcp"
   queue.type="LinkedList" queue.filename="fwd_stor01"
   queue.saveOnShutdown="on" action.resumeRetryCount="-1")
^O Write Out (save)    ^X Exit
```

`*.*` = every line · `omfwd` = output module, forward to `192.168.60.20` port 514 over TCP · `queue…` = if stor01 is down, lines wait **on the sender** · `saveOnShutdown` = kept over a restart · `resumeRetryCount="-1"` = never stop trying

db01 – PostgreSQL (one line, changed with sed)

`log_destination = 'stderr,syslog'`	stderr = PostgreSQL's own file (stays) · syslog = **also** to rsyslog → reaches stor01

3 · All commands in order

Server	Command	GOOD answer
stor01	`echo "192.168.60.30 db01" | sudo tee -a /etc/hosts` `ping -c 2 db01`	**0% packet loss**
stor01	`sudo mkdir -p /var/log/remote` `sudo chown syslog:adm /var/log/remote`	**ls -ld shows syslog adm**
stor01	`sudo nano /etc/rsyslog.d/10-receive.conf`	type the receive file (section 2)

stor01	`sudo rsyslogd -N1`
	`sudo systemctl restart rsyslog && systemctl is-active rsyslog`
	`sudo ss -tlnp | grep 514`
	`sudo ufw status`
		Bye. · active · LISTEN 0.0.0.0:514 rsyslogd · inactive

app01	`sudo nano /etc/rsyslog.d/90-forward.conf`
	`sudo rsyslogd -N1`
	`sudo systemctl restart rsyslog && systemctl is-active rsyslog`
	`logger -t ticket10 "hello from app01"`
		Bye. · active

stor01	`sudo tail /var/log/remote/app01/ticket10.log`	**app01 ticket10: hello from app01**
app01 → stor01	`~/price-update.sh` `sudo tail -3 /var/log/remote/app01/price-update.log`	**started · done: 3 prices changed**

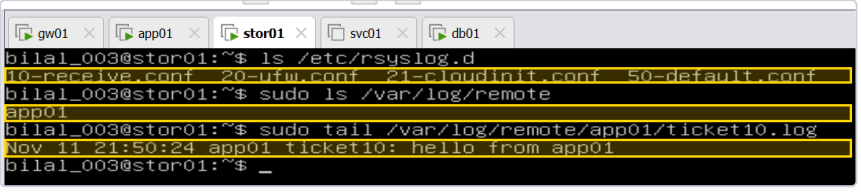
db01	`sudo nano /etc/rsyslog.d/90-forward.conf  # same file`
	`sudo rsyslogd -N1`
	`sudo systemctl restart rsyslog && systemctl is-active rsyslog`
	`logger -t ticket10 "hello from db01"`
		Bye. · active · on stor01: db01 ticket10: hello from db01

db01	`sudo sed -i "s/^#\?log_destination.*/log_destination = 'stderr,syslog'/" /etc/postgresql/14/main/postgresql.conf` `sudo systemctl reload postgresql@14-main`	**log_destination = 'stderr,syslog' · DB keeps running**
app01 → stor01	`psql -h db01 -U shopapp -d shop -c "SELECT pg_sleep(3);"` `sudo tail -3 /var/log/remote/db01/postgres.log`	**duration: 3074 ms statement: SELECT pg_sleep(3);**

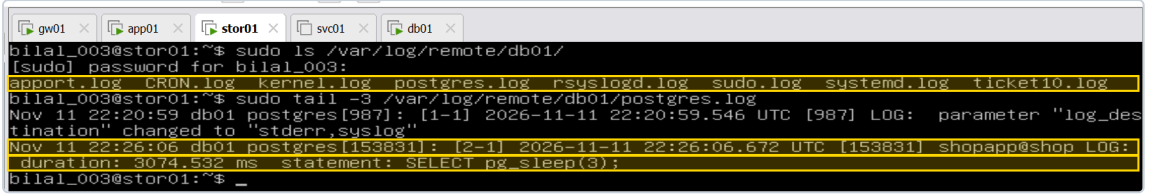
Test D	`stor01: sudo systemctl stop syslog.socket rsyslog`
	`app01:  logger -t ticket10 "sent while stor01 was down"`
	`stor01: sudo grep "was down" /var/log/remote/app01/ticket10.log`
	`stor01: sudo systemctl start rsyslog ; sleep 60`
	`stor01: sudo grep "was down" /var/log/remote/app01/ticket10.log`
		1st grep: nothing (not there during the outage)
		2nd grep: the line, with its original time → no line lost

4 · The proof

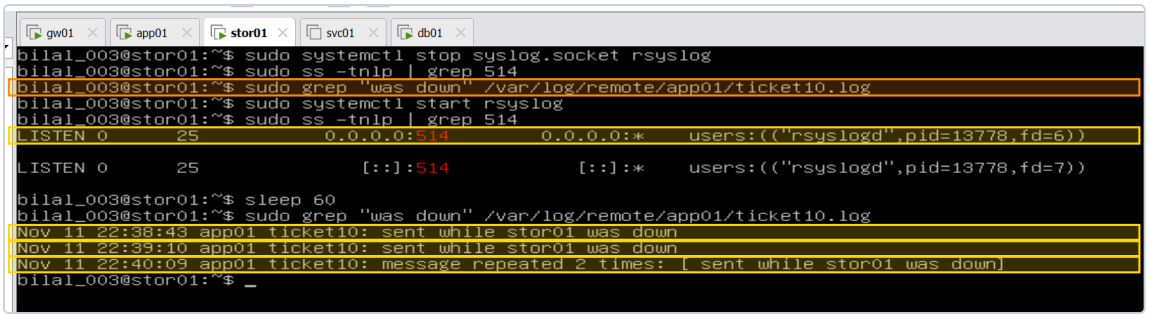
app01's line on stor01



Ticket #9's slow-query line from db01 on stor01



Test D – written while stor01 was down, arrived later



Final paragraph. stor01 is the central log server: `10-receive.conf` makes rsyslog listen on TCP 514 (imtcp) and write every incoming line to `/var/log/remote/<server>/<program>.log` (omfile). app01 and db01 have the same `90-forward.conf`: every line (`*.*`) goes to stor01 over TCP (omfwd), with a queue on the sender. PostgreSQL writes its own file, so on db01 one line (`log_destination = 'stderr,syslog'`) plus a reload sends its log to rsyslog too. Result on stor01, without logging in to app01 or db01: the test lines, the price script's started/done (ticket #8) and the slow-query line (ticket #9). When stor01 was down, app01 kept its line in the queue and sent it later with its original time – no line lost. **Next:** 5.5 persistent journal · 5.6 logrotate · Phase 6 auditd (who touched a log file).